Remember how instance variables work in Ruby: the first mention of an @-prefixed variable creates the instance variable *in the current object*, self.

For a mixin, this means the module you mix into your client class may create instance variables in the client object and may use attr_reader and friends to define accessors for these instance variables. For instance, the Observable module in the following example adds an instance variable @observer_list to any class that includes it:

tut_modules/observer_impl.rb

```
module Observable
  def observers
    @observer_list ||= []
  end

  def add_observer(obj)
    observers << obj
  end

  def notify_observers
    observers.each { |o| o.update }
  end
end
```

However, this behavior exposes us to a risk. A mixin's instance variables can clash with those of the host class or with those of other mixins. The example that follows shows a class that uses our Observable module but that unluckily also uses an instance variable called @observer_list. At runtime, this program will go wrong in some hard-to-diagnose ways:

tut_modules/observer_impl_eg.rb

```
require_relative "observer_impl"

class TelescopeScheduler
  # other classes can register to get notifications
  # when the schedule changes
  include Observable

  def initialize
    @observer_list = []  # folks with telescope time
  end

  def add_viewer(viewer)
    @observer_list << viewer
  end

  # ...
end
```

For the most part, mixin modules don't use instance variables directly—they use accessors to retrieve data from the client object. But if you need to create a mixin that has to have its own state, ensure that the instance variables have unique names to distinguish them from any other mixins in the system (perhaps by using the module's name as part of the variable name). Alternatively, the module could use a module-level hash, indexed by the current object ID, to store instance-specific data without using Ruby instance variables:

tut_modules/state_eg.rb

```
module Test
  def self.states
    @states ||= {}
```